# 26-Arch Lab1 实验报告

## 1. 基本信息

- 姓名：潘孝圆
- 学号：24300240128
- 课程：26-Arch
- 实验：Lab1（64 位算术 CPU）

## 2. 实验目标与要求

本实验要求完成一个支持 64 位整数算术的 CPU 核心，并通过给定 difftest。

必做指令：

- `addi, xori, ori, andi`
- `add, sub, and, or, xor`
- `addiw, addw, subw`

选做指令（本次已完成）：

- `mul, div, divu, rem, remu`
- `mulw, divw, divuw, remw, remuw`

通过标准：

- `make test-lab1` 出现 `HIT GOOD TRAP`
- 选做指令 `make test-lab1-extra` 出现 `HIT GOOD TRAP`

## 3. 总体设计说明

### 3.1 数据通路与阶段组织

当前 `core.sv` 的主路径按“取指-译码/执行-写回”组织，使用了明确的阶段寄存器：

- IF 侧：
  - `pc`
  - `if_pending`（是否有未完成取指请求）
  - `if_req_addr`
- IF/ID 缓冲：
  - `if_id_valid`
  - `if_id_pc`
  - `if_id_instr`
- EX/WB 缓冲：
  - `ex_wb_valid`
  - `ex_wb_wen`

    - `ex_wb_rd`
    - `ex_wb_data`
    - `ex_wb_pc`
    - `ex_wb_instr`

由于本实验不涉及数据存储器读写，`dreq` 固定为 `0`，重点在算术/逻辑执行与正确提交。

## 3.2 指令获取与 IF/ID 控制

取指逻辑要点：

1. 当 `!if_pending && !if_id_valid` 时发起取指请求。
2. `iresp.data_ok` 到达后，填充 `if_id_pc/if_id_instr` 并置 `if_id_valid=1`。
3. 当下游声明消耗（`id_consume=1`）时，清除 `if_id_valid`。

这样可以保证 IF 与下游解耦，不会重复消费同一条指令。

## 3.3 译码与执行控制信号

译码输出的关键控制信号：

- `id_valid`
- `id_wen`
- `id_use_imm`
- `id_is_word`
- `id_is_md`（是否 M 扩展）
- `id_alu_op`（基础 ALU 选择）
- `id_md_op`（M 扩展子操作码）

基础 ALU 支持 `ADD/SUB/AND/OR/XOR`，操作数来自：

- `ex_op1 = rs1_val`
- `ex_op2 = id_use_imm ? imm_i : rs2_val`

`W` 类指令统一使用 32 位结果再符号扩展回 64 位（`sext32`）。

## 3.4 寄存器堆与写回

- 寄存器堆 `gpr[31:0]` 在写回阶段更新，`x0` 强制保持 0。
- 写回数据从 `ex_wb_*` 输出到 `wb_*`，再写寄存器并同步给 difftest 提交接口。

# 4. 选做指令实现与乘除法自动机原理

## 4.1 本次代码实现方式

本次实现已完成全部 10 条 M 扩展指令，核心逻辑在：

- `mul_u64`（移位加法）

- udivrem64（恢复除法）
- MD_* 的 case 结果生成

并且未使用 *、/、% 运算符。

当前代码里，mul_u64/udivrem64 使用 for 循环在组合逻辑中一次性计算结果，功能上正确，能通过实验测试。

## 4.2 乘法自动机原理（多周期可综合思路）

实验文档强调的“正确工程实现”是多周期状态机（FSM），其原理如下。

状态定义（示例）：

- MUL_IDLE：等待乘法指令
- MUL_PREP：装载操作数，处理符号位
- MUL_ITER：逐位迭代
- MUL_FIX：符号修正、mulw 截断与扩展
- MUL_DONE：结果写回，释放 busy

关键寄存器：

- mul_mcand（被乘数移位寄存器）
- mul_mplier（乘数移位寄存器）
- mul_acc（累加器）
- mul_cnt（0~63 迭代计数）
- mul_sign（结果符号）

每拍迭代规则（MUL_ITER）：

1. 若 mul_mplier[0] == 1，则 mul_acc += mul_mcand
2. mul_mcand <<= 1
3. mul_mplier >>= 1
4. mul_cnt -= 1

结束条件：

- mul_cnt==0 后进入 MUL_FIX/MUL_DONE
- 对有符号乘法根据 mul_sign 做补码修正
- mulw 取低 32 位并符号扩展到 64 位

这样每拍只经过一小段组合逻辑，时序压力明显小于一次性 64 位乘法大组合网。

## 4.3 除法/取余自动机原理（恢复除法）

同样使用多周期 FSM，状态示例：

- DIV_IDLE
- DIV_PREP
- DIV_ITER
- DIV_FIX

- `DIV_DONE`

关键寄存器：

- `div_rem`（余数寄存器，通常 65 位）
- `div_quot`（商寄存器）
- `div_divisor`
- `div_cnt`
- `div_sign_q`（商符号）
- `div_sign_r`（余数符号）

每拍迭代（恢复除法）：

1. `{div_rem, div_quot}` 左移 1 位
2. 比较 `div_rem` 与 `div_divisor`
3. 若 `div_rem >= div_divisor`：
    - `div_rem -= div_divisor`
    - `div_quot[0] = 1`
4. `div_cnt -= 1`

边界语义（RISC-V 规范）：

- 除零：
    - `div/divu/divw/divuw` 返回全 1
    - `rem/remu/remw/remuw` 返回被除数
- 溢出（`INT_MIN / -1`）：
    - 商返回 `INT_MIN`
    - 余数返回 0

最终在 `DIV_FIX` 按有符号/无符号、64/32 位规则处理符号与扩展。

## 4.4 多周期自动机与流水线接口关系

如果按 FSM 落地，通常需要：

1. `md_busy`：乘除法单元忙信号
2. `id_stall`：当解码到 M 指令且 `md_busy=1` 时阻塞前级
3. `ex_valid_hold`：冻结正在等待结果的指令上下文（`rd/pc/instr`）
4. `md_done`：结果有效时进入 WB 提交

即：M 指令在 EX 占用多个周期，其他阶段按策略 stall 或旁路推进。

本实验当前代码是"功能先通过版"，算法与 FSM 一致，但计算在组合里一次完成；后续上板/综合时建议替换为上述多周期 FSM 结构。

# 5. 数据冒险处理说明

当前版本未实现完整的前递网络与显式 stall 状态机，属于基础可运行方案。
在给定 lab1/lab1-extra 测试下可通过。

若继续完善到更标准流水实现，建议：

- 增加 RAW 检测
- 加入 `EX->ID` / `WB->ID` 前递选择器
- 对不可前递场景引入精确 stall

## 6. Difftest 与 Trap 对接

已接入以下模块：

- `DifftestInstrCommit`
- `DifftestArchIntRegState`
- `DifftestTrapEvent`
- `DifftestCSRState`

Trap 事件处理：

- `trap_valid = dt_valid && (dt_instr == 32'h0005006b)`
- `trap_code = gpr[10][2:0]`
- 维护 `cyc_cnt`、`instr_cnt` 并上报

这保证了测试结束时 DUT 能与参考模型一致退出。

## 7. 测试过程与结果

### 7.1 必做测试

命令：

```
make test-lab1
```

结果摘要：

- `Core 0: HIT GOOD TRAP at pc = 0x80010004`
- `instrCnt = 16385, cycleCnt = 65545`

### 7.2 选做测试

命令：

```
make test-lab1-extra
```

结果摘要：

- `Core 0: HIT GOOD TRAP at pc = 0x8002001c`
- `instrCnt = 32775, cycleCnt = 131105`

结论：`lab1` 与 `lab1-extra` 均通过。

## 8. 调试记录（关键问题）

1. **提交流断裂问题**
   decode 默认值曾导致 `id_valid` 被清零，修正为继承 `if_id_valid`，提交恢复连续。
2. **Trap 收尾问题**
   初期 `DifftestTrapEvent.valid` 未正确触发，导致参考模型先结束而 DUT 继续提交，出现 `this_pc different`。补齐 Trap 触发后解决。
3. **W 类结果符号扩展问题**
   增加 `sext32` 并统一 `W` 类写回路径，修复 `addiw/addw/subw` 及 `mulw/divw/remw` 的高位错误。

## 9. AI 使用说明

本实验在本人独立编码与调试的基础上，使用了 **OpenAI Codex（GPT-5.3）** 进行辅助，主要包括：

- 日志分析与问题定位建议
- 边界条件核对（`div/rem` 语义）
- 变量/信号命名规范化建议（如 `if_/id_/ex_/wb_/dt_` 前缀一致性）
- 乘除法自动机与流水线相关原理讲解（用于辅助理解与设计取舍）
- 报告文本整理

核心设计、代码实现、调试验证与最终提交均由本人独立完成，AI 仅作为辅助工具。

## 10. 总结与后续改进

本次 Lab1 已完成全部必做与选做指令并通过测试。
后续可继续优化：

1. 将乘除法从组合迭代升级为多周期 FSM（更符合综合与时序目标）
2. 增加前递与 stall 机制，完善流水线冒险处理
3. 补充对异常/中断和 CSR 的更完整支持